

计算机网络实验报告

实验题目：抓包分析

WWW 服务器、DNS 服务器和 FTP 服务器的安装、配置与使用

HTTP、TCP 协议分析及基于 TCP 协议的 Socket 通信编程

一. 实验目的

- (1) 验证 HTTP 等协议下的请求报文和响应报文信息。
- (2) 掌握 Web 服务器的安装、配置与使用方法。
- (3) 掌握 DNS 服务器的安装、配置与使用方法。
- (4) 掌握 FTP 服务器的安装、配置与使用方法。
- (5) 验证 HTTP 协议的报文结构，分析 HTTP 协议的工作方式。
- (6) 验证 TCP 协议的报文段结构，分析 TCP 协议的工作方式。
- (7) 编程实现基于 TCP 协议的 Socket 通信。

二. 实验环境与器材

实验环境：实验室机房。

实验平台：硬件平台：机房服务器、thinkpad

软件平台：Windows 2016 操作系统、Wireshark 3.6.6

三. 实验过程描述

(一) Web 服务器的安装、配置与使用方法

- ①点击“服务管理器”->打开服务管理器窗口->点击“添加角色和功能”



②连续点击三个“下一步”->到达选择服务器角色页面，勾选“web 服务器 iis”->下一步->下一步



③在角色服务窗口选择“IP 和域限制”和“Windows 身份验证”->下一步->最后点击确认安装，然后就ok了。

④转跳到本报告的实验过程描述（二）DNS 服务器的安装、配置与使用方法部分，先把那部分做了来，把网连上。

⑤再次打开 IIS 管理器，右键网站选择添加网站，物理路径指向网页源码的源文件夹，ip 地址还是前面填过很多次那个，再填一遍。

添加网站

网站名称(S): index 应用程序池(L): index 选择(E)...

内容目录

物理路径(P): C:\Users\Administrator\Desktop\index ...

传递身份验证

连接为(C)... 测试设置(G)...

绑定

类型(T): http IP 地址(I): 192.168.190.201 端口(O): 80

主机名(H):

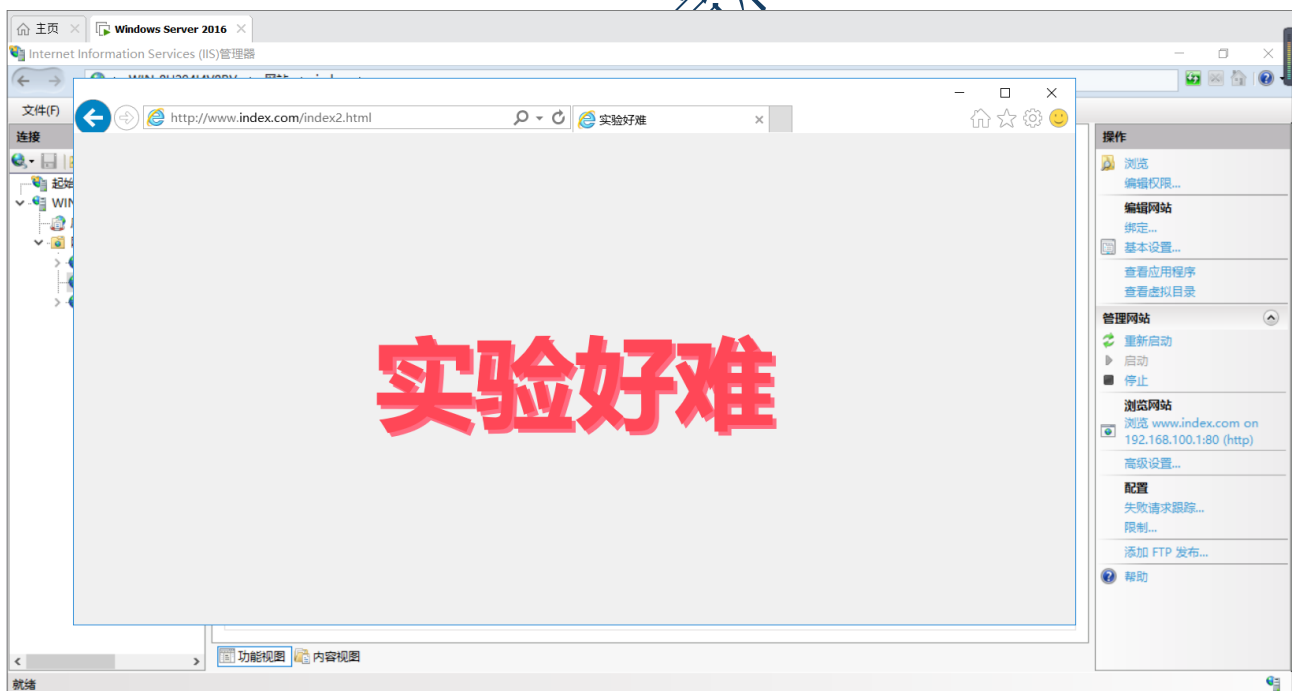
示例: www.contoso.com 或 marketing.contoso.com

☒ 立即启动网站(M)

确定 取消

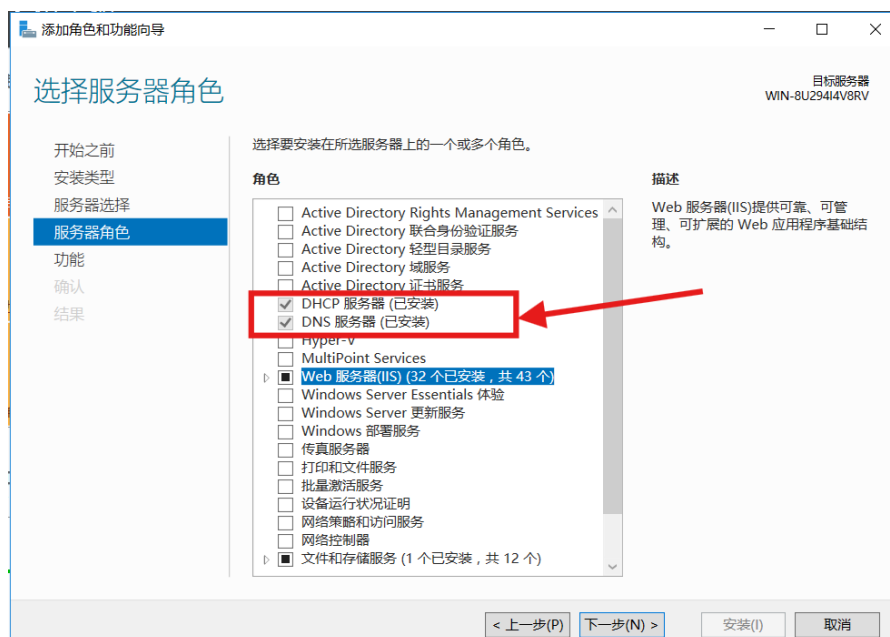
禁抄袭!

⑥在上一步骤的物理路径中，我写了个html放进去，这个html是可以打开的。点击管理网站中的“启动”->“浏览网站”下的链接点进去->访问成功：

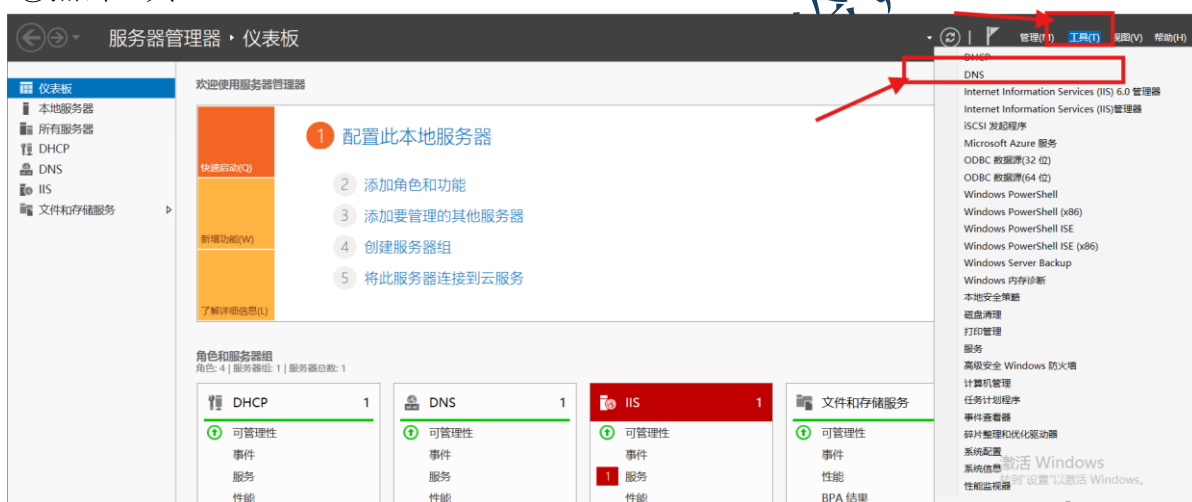


(二) DNS 服务器的安装、配置与使用方法

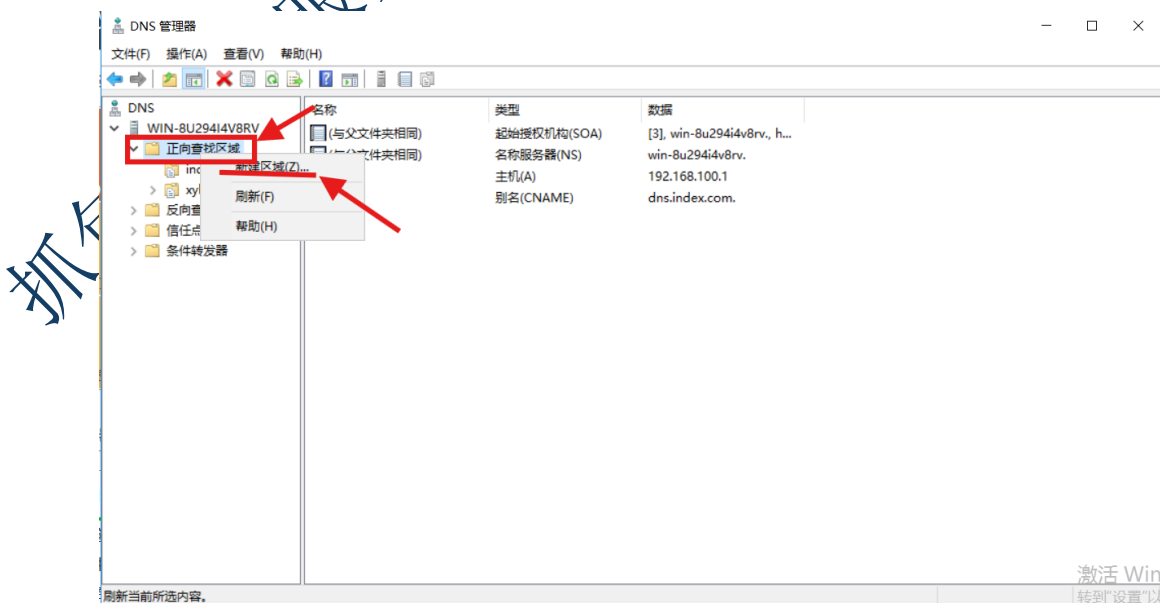
①点开“服务管理器”->点击“DNS 服务器”->点击下一步，直到安装->安装完成



②点击工具->“DNS”



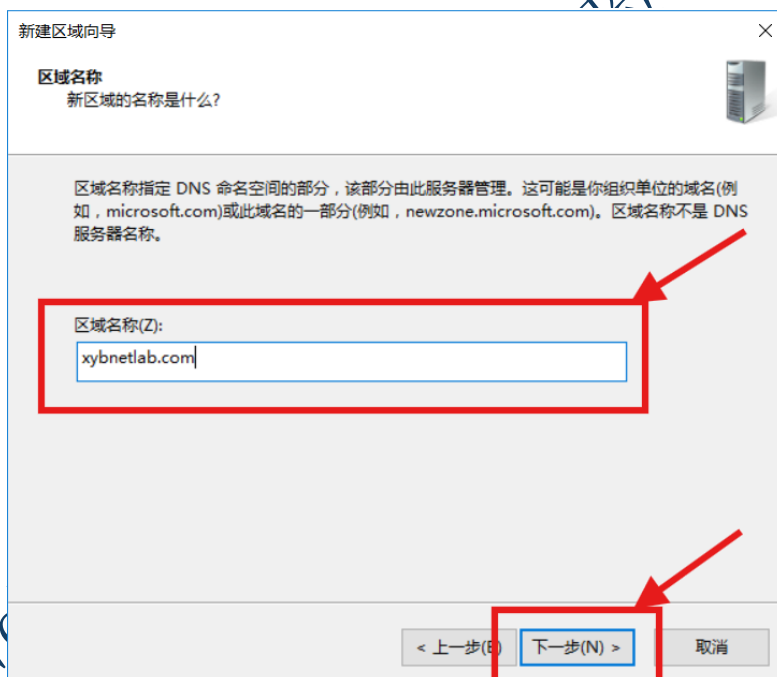
③右击“正向查找区域”，选择“新建区域”



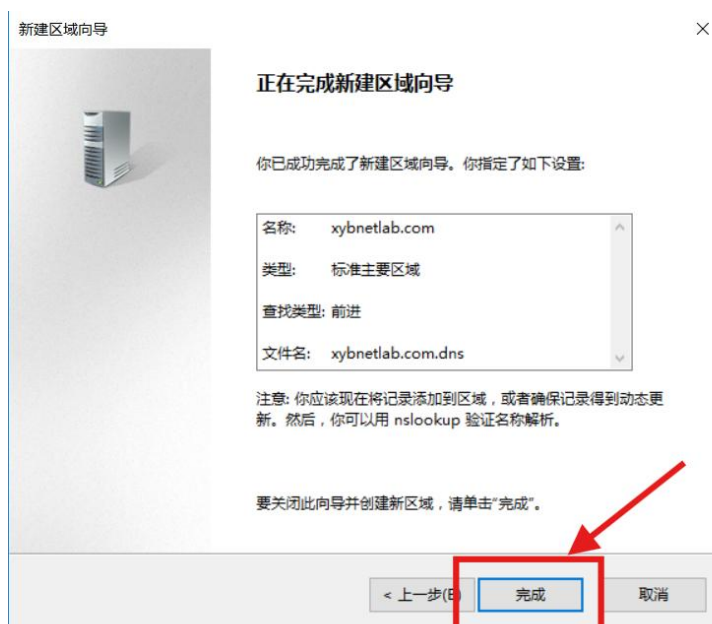
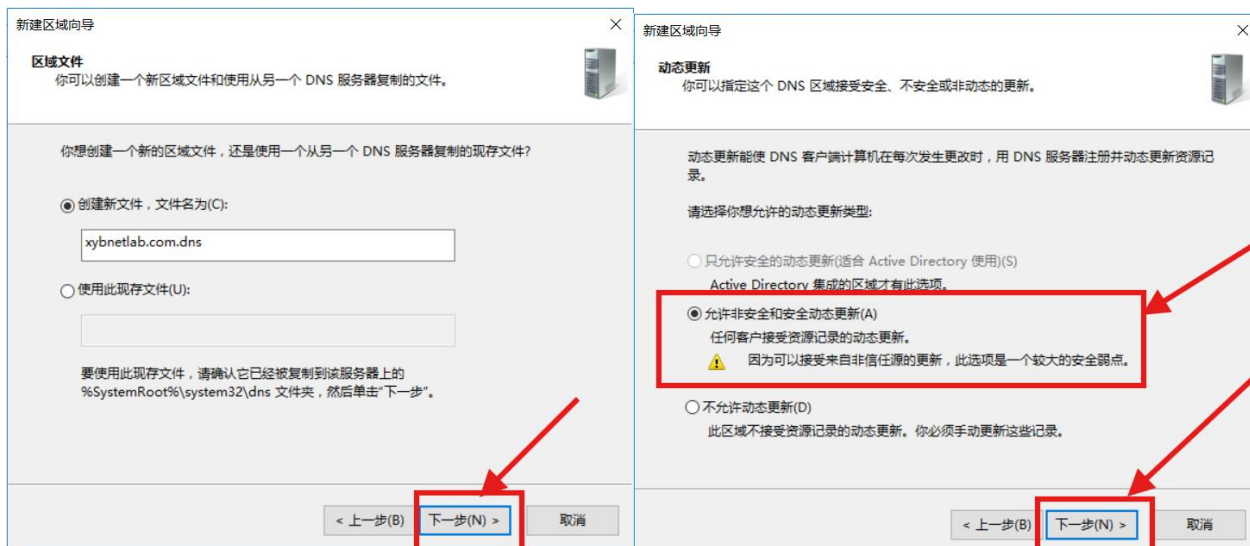
④选择“主要区域”，点击下一步。



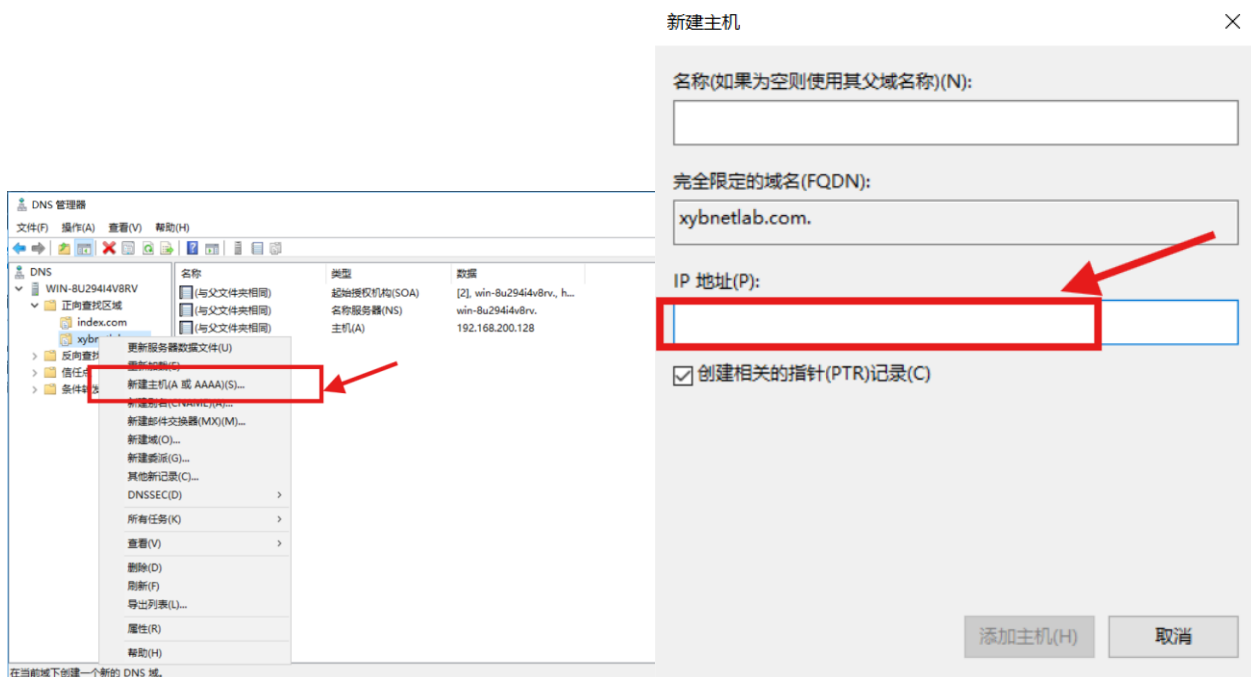
⑤设置区域名称，如“xybnetlab.com”



⑥点击下一步:



⑦点击新建主机--填写服务器的 ip 地址：



⑧右键反向查找区域，点击新建区域->选择主要区域。



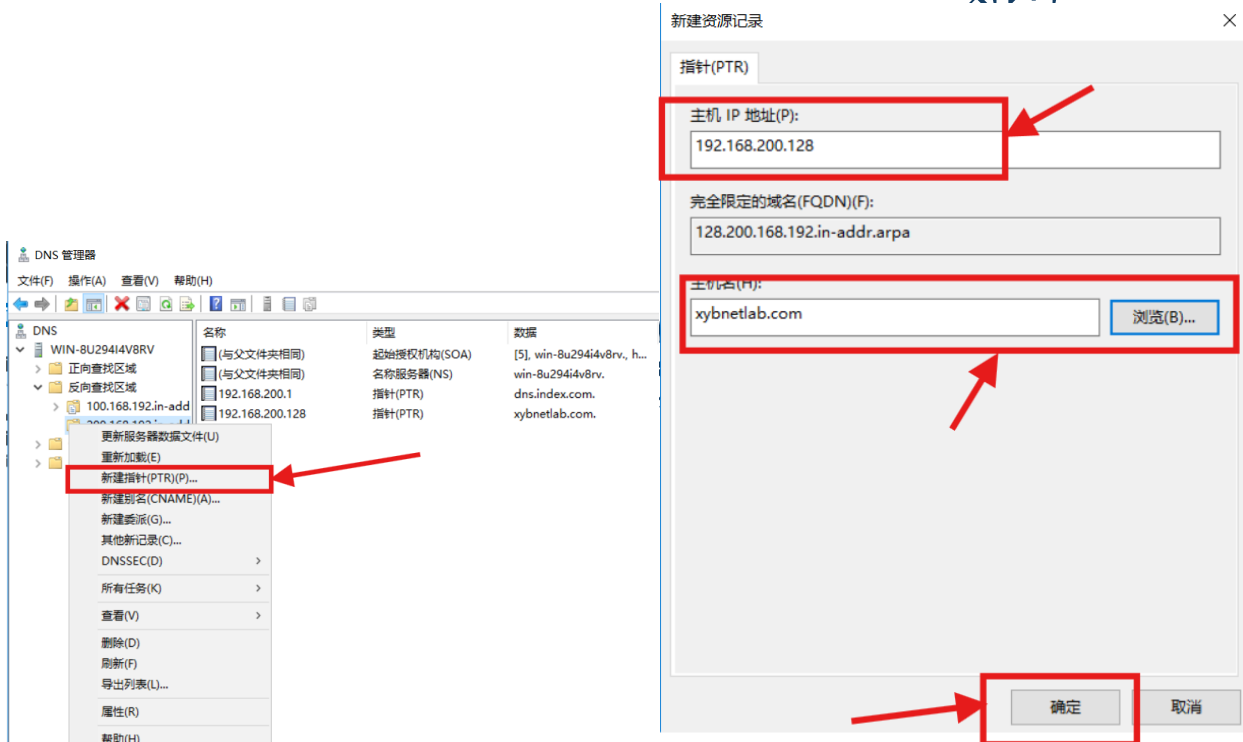
⑨选择 IPv4 反向查找区域->填写网络 id



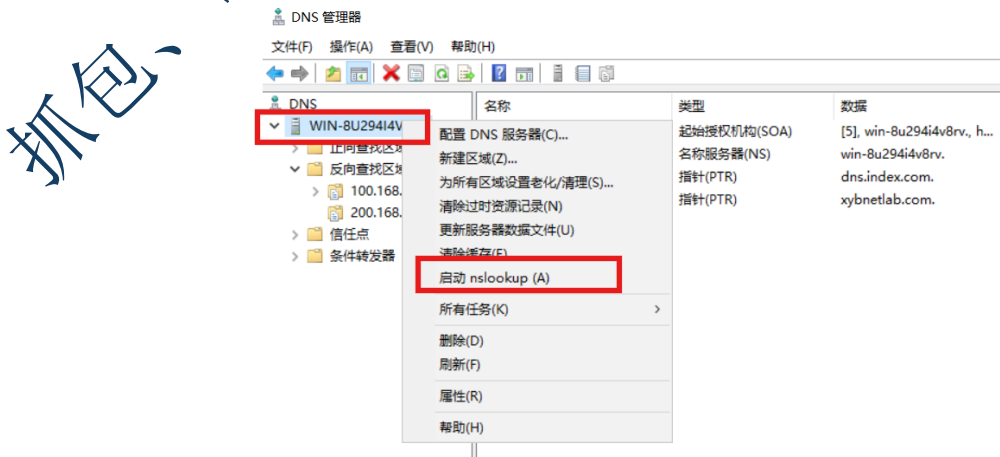
⑩连续点击下一步->完成



11. 右击刚刚建好的区域->新建指针->填写刚刚写得主机地址->选择主机->确定



12. 右键选择启动“nslookup”



13. 测试:


```
> xybnetlab.com
服务器: UnKnown
Address: fe80::c8e9:ec82:3799:3950

名称: xybnetlab.com
Address: 192.168.200.128
```

14.设置 IPv4 DNS 服务器，即虚拟机的 IP 地址：打开网络和共享中心->点击 Ethernet0

查看基本网络信息并设置连接

查看活动网络

网络

专用网络

访问类型: Internet

连接: Ethernet0

未识别的网络

公用网络

访问类型: Internet

连接: Ethernet0

更改网络设置



设置新的连接或网络

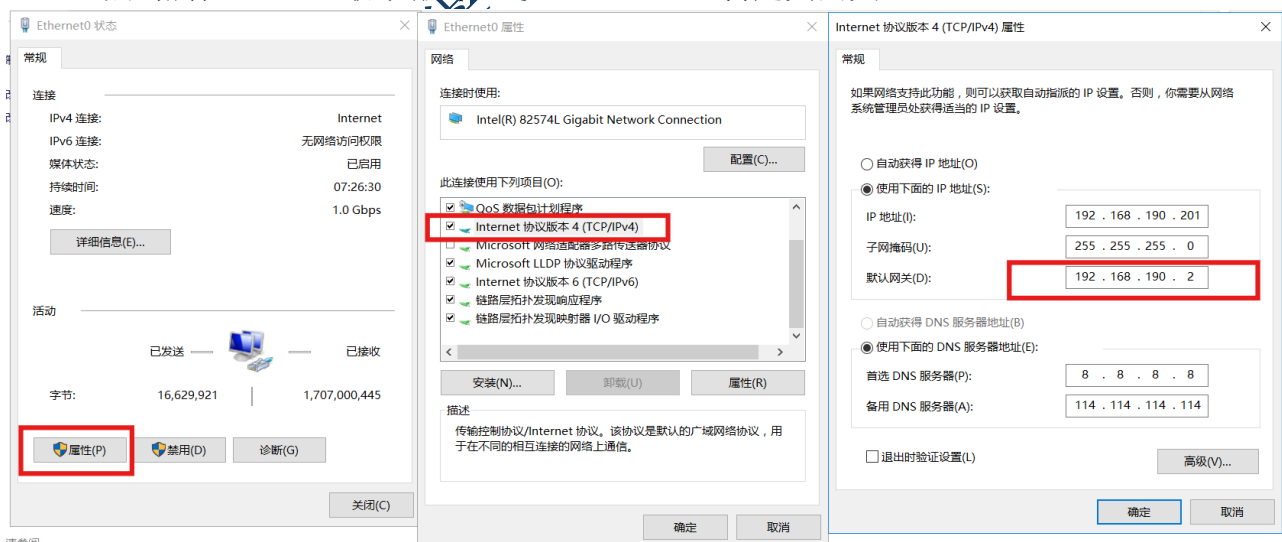
设置宽带、拨号或 VPN 连接；或设置路由器或接入点。



问题疑难解答

诊断并修复网络问题，或者获得疑难解答信息。

15.点击属性-Internet 协议版本 4 (TCP/IPv4) ->自定义网关

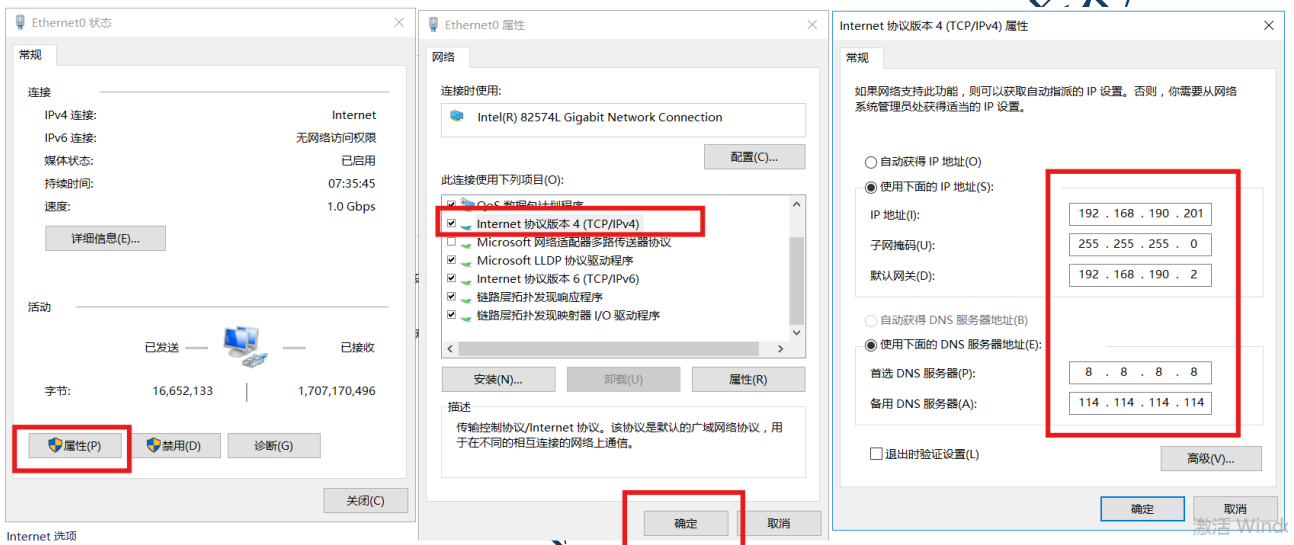


16.在本机上设置 VMware Network Adapter VMnet8 网络设置（因为虚拟机连接 NAT 网络）

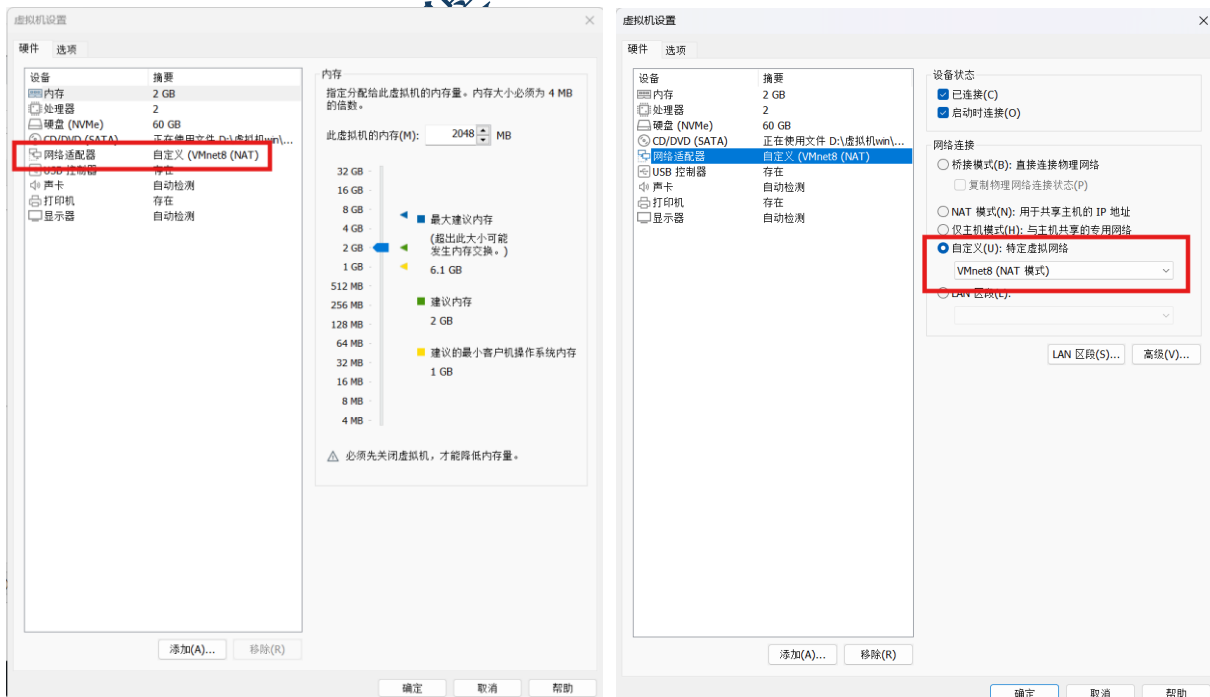
查看基本网络信息并设置连接



17.和前面类似的，再填一遍：

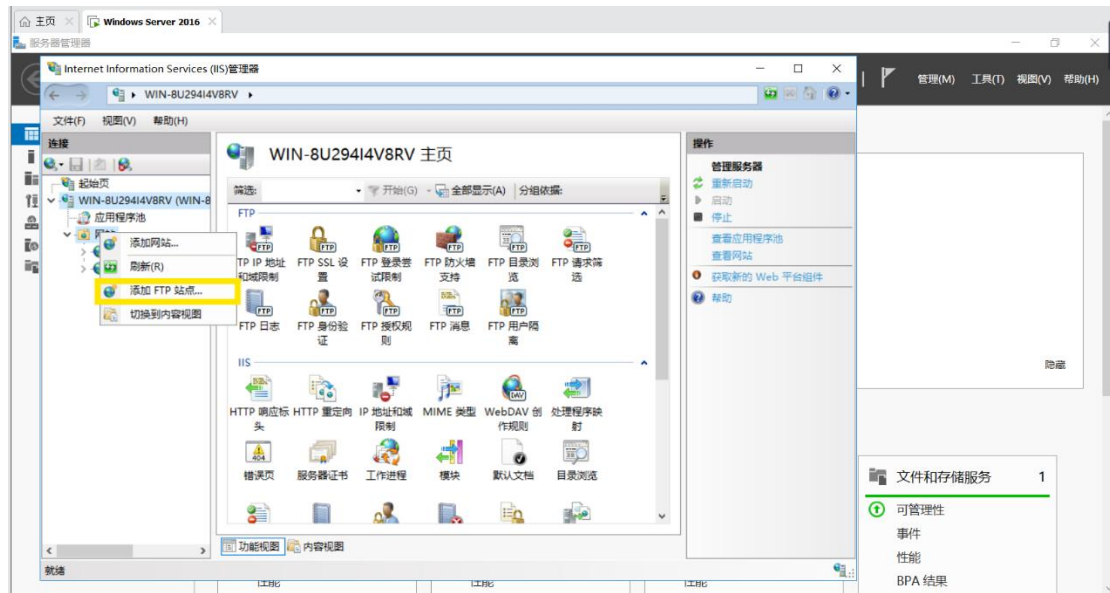


18.回到虚拟机，点击网络适配器->选择 Vmnet8(NAT 模式)->连上网就成功了

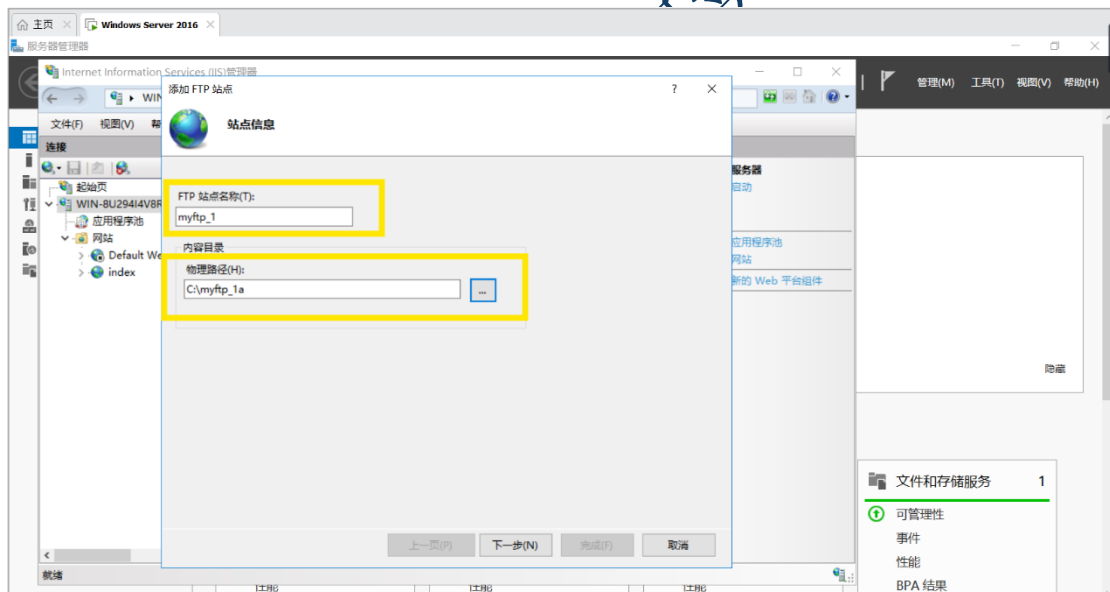


(三) FTP 服务器的安装、配置与使用方法

①在 IIS 管理器中右键点击“添加 FTP 站点”。

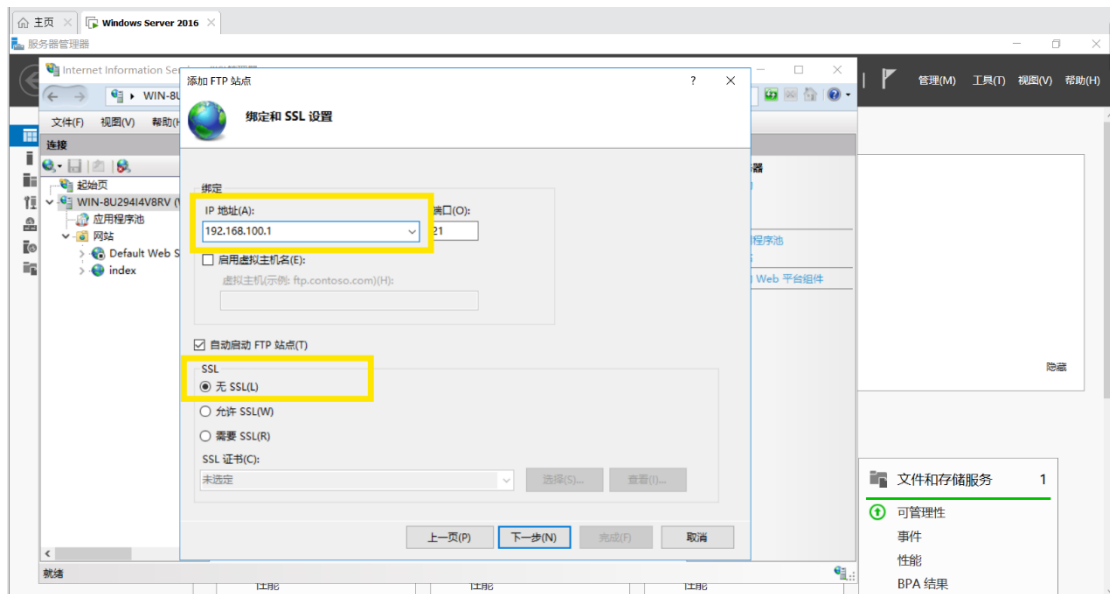


②填写“FTP 站点名称”和物理路径，并点击下一步。

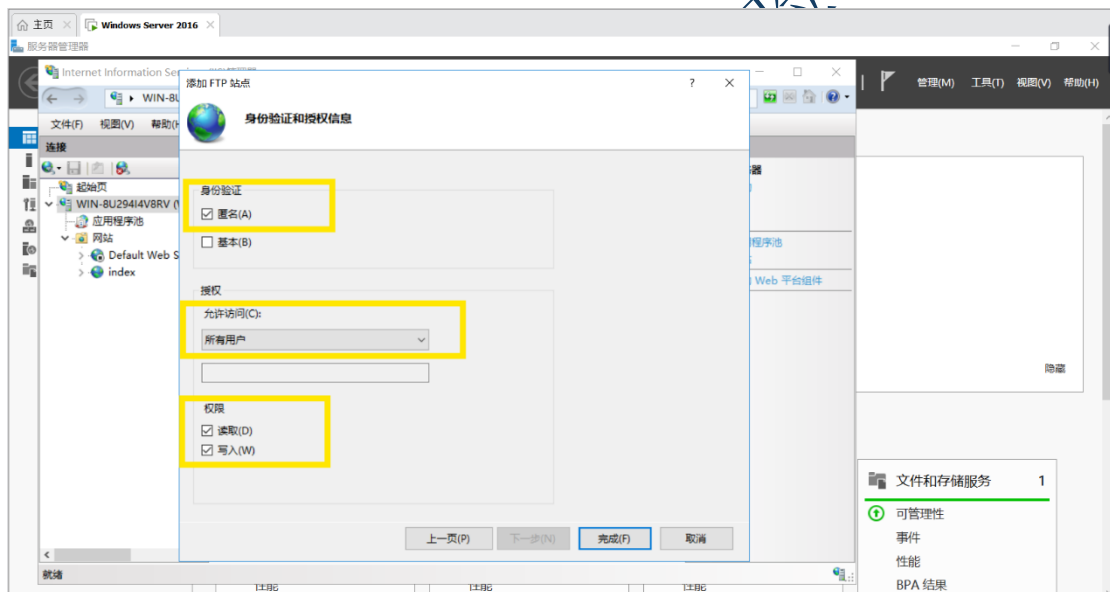


③设置绑定和 SSL 设置，填写“IP 地址”，将 SSL 处设置为“无 SSL”。

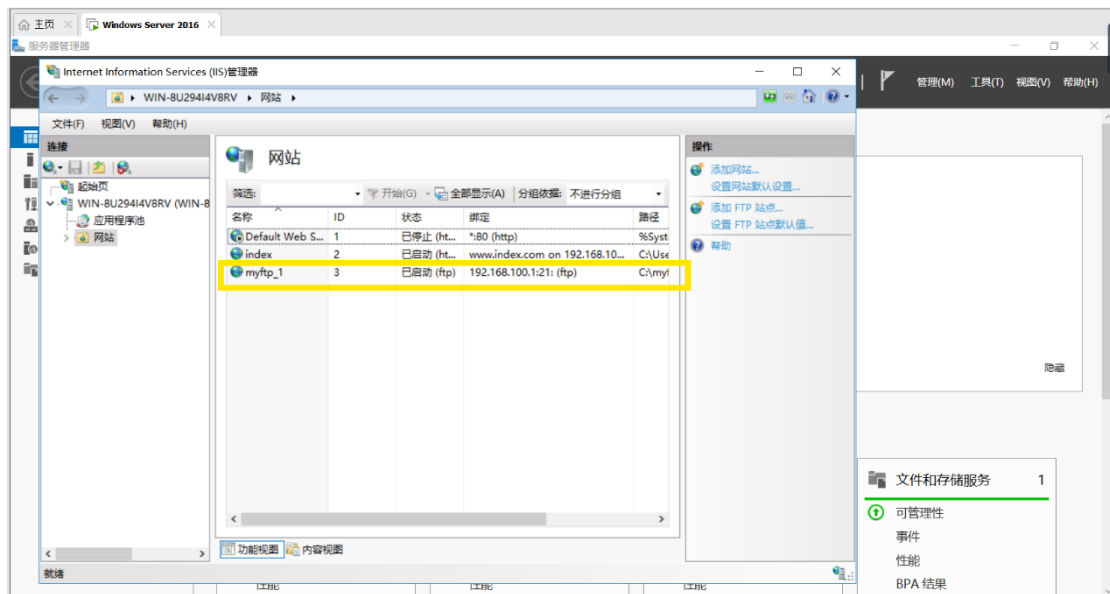
抓包



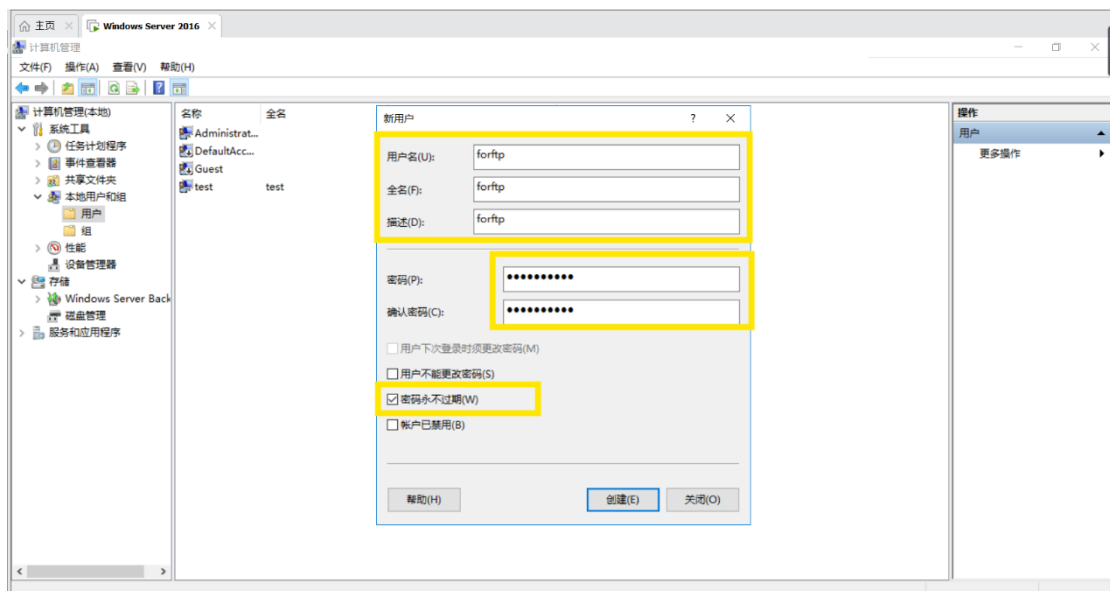
④ 设置“身份验证和授权信息”，身份验证勾选为“匿名”，授权为“所有用户”，权限勾选上“读取”与“写入”->点击完成->显示成功。



抓包、Web

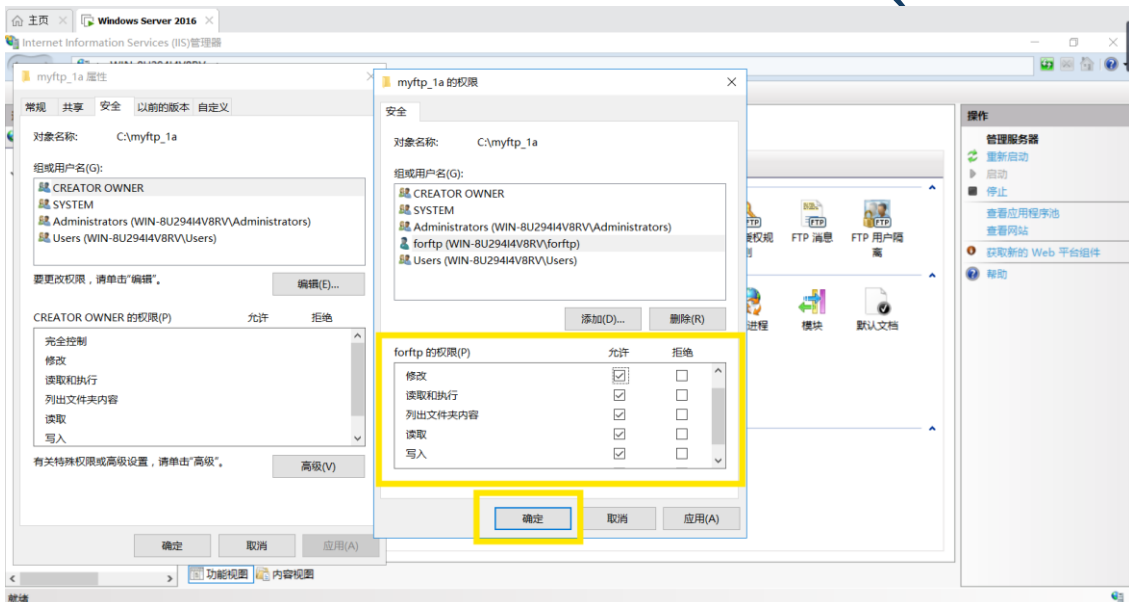
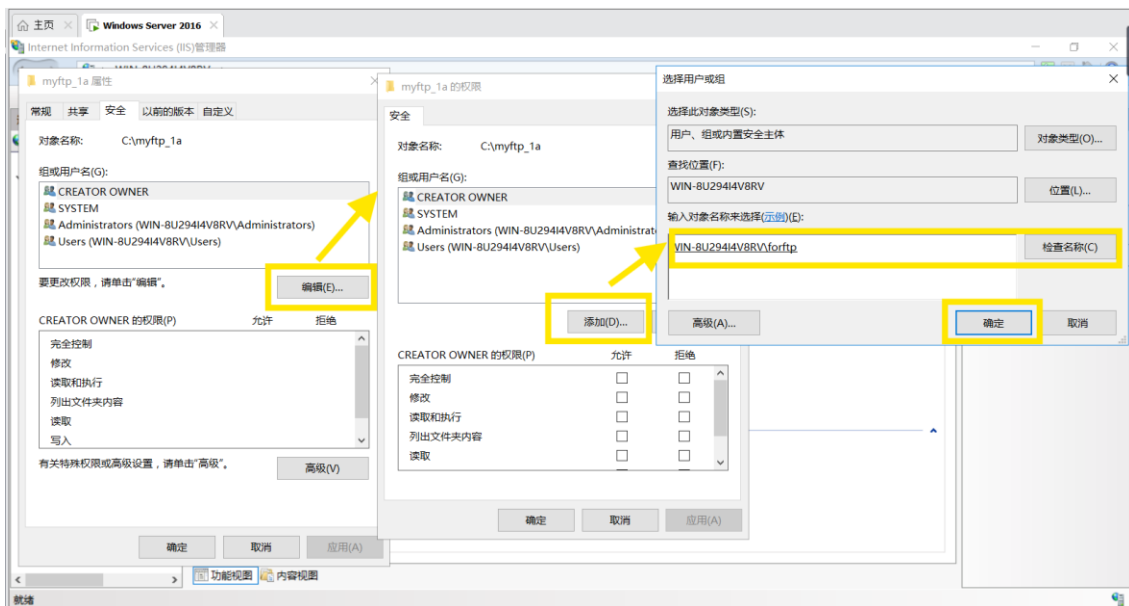


⑤在此处，我设置了一个用于测试 ftp 的用户 forftp

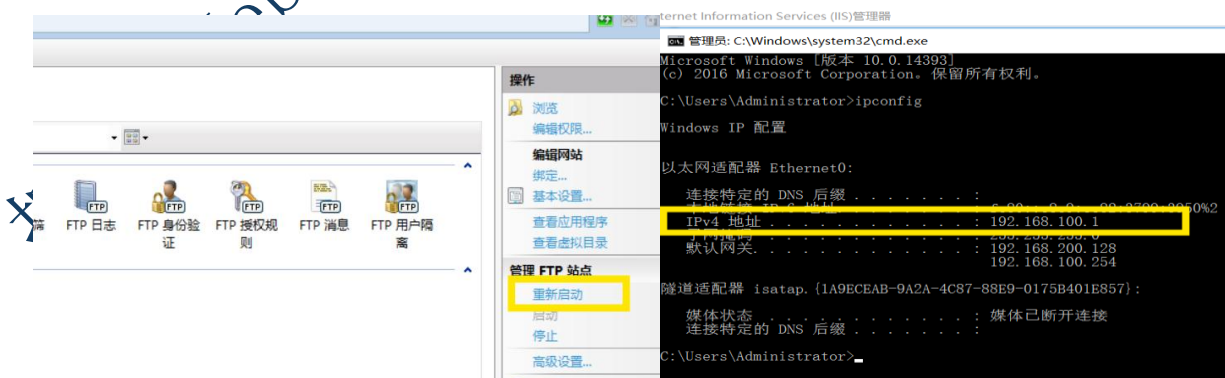


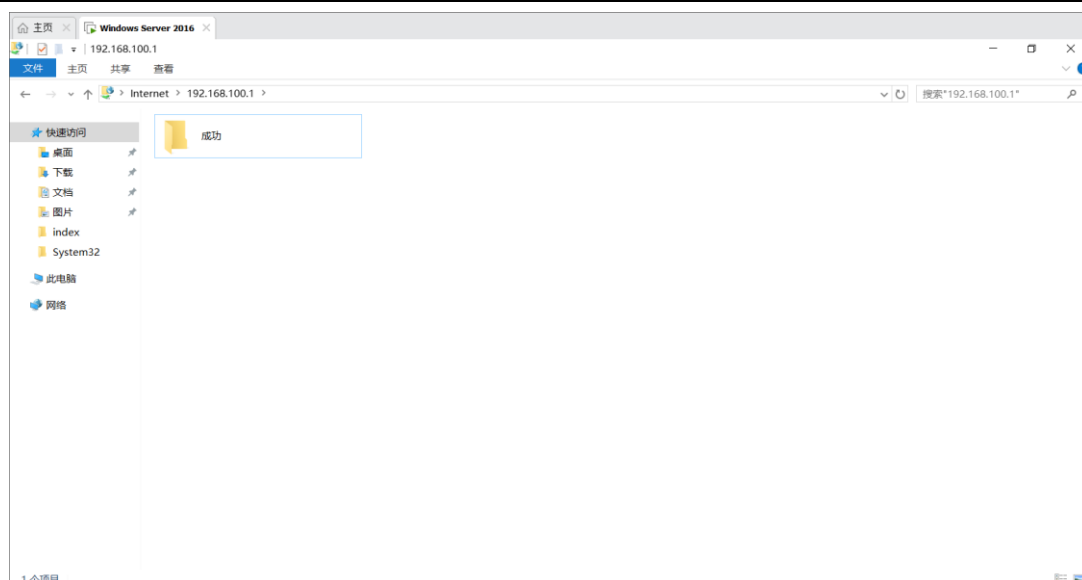
⑥设置 ftp 相关的权限，将刚刚创建的 forftp 这个用户加到里面。

抓包、Web



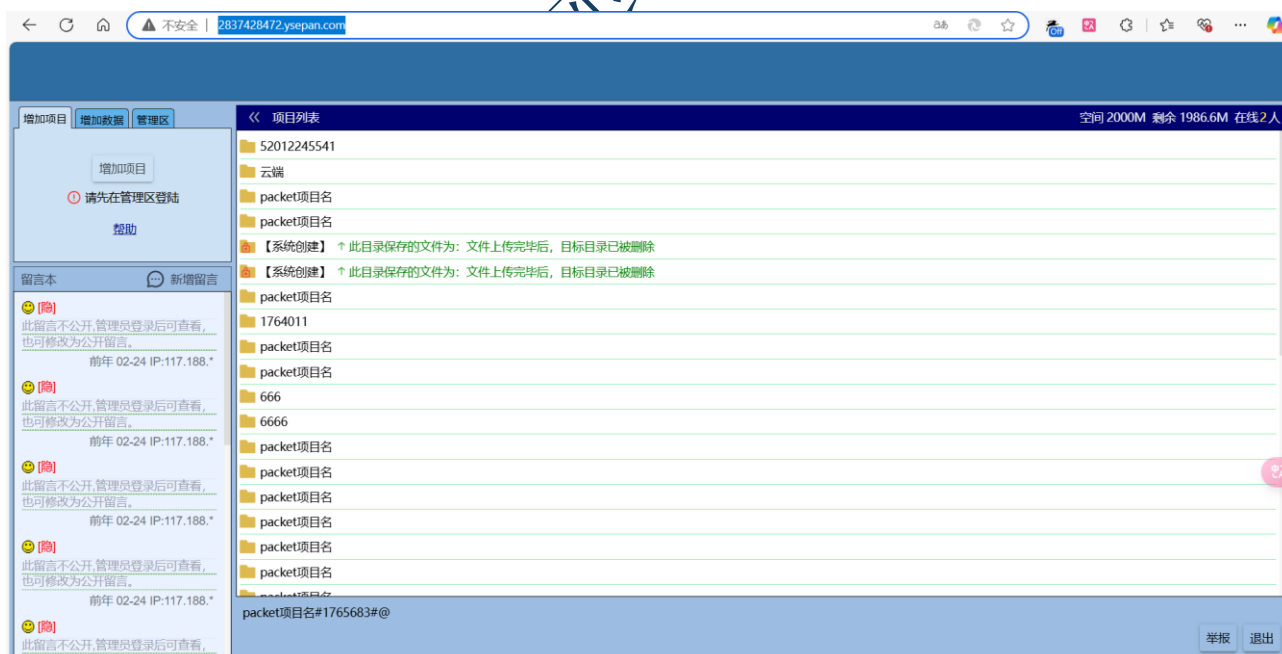
⑦重启站点->查找对应的 ip 地址->成功进入站点





(四) 访问任何一个 Web 站点，通过 Wireshark 软件捕获你的 Web 浏览器与上述站点之间交换的协议报文（如 HTTP 协议下的请求报文和响应报文、TCP 协议下的请求报文段和响应报文段），并结合课上积累的知识，尝试分析 HTTP 协议和 TCP 协议的工作方式。

访问站点：网上随便找的一个：<http://2837428472.ysepan.com/>

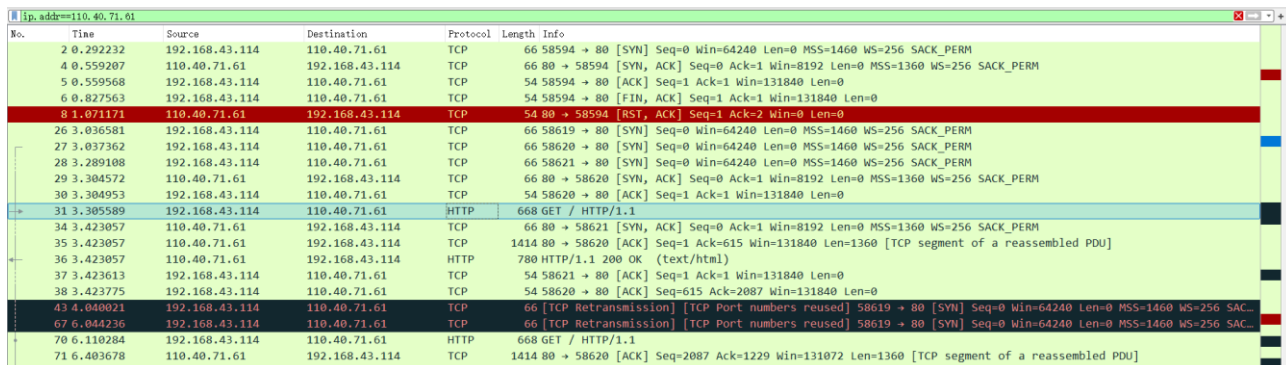


打开这个网站。然后我们在 cmd 中查找到这个网站的 ip 地址，如图 1 所示：



图 1 查找网站的 ip 地址

打开 wireshark 之后，刷新这个网站，然后筛选出该网站的相关数据，其与服务器的数据交换如图 2 wireshark 捕获所示：



No.	Time	Source	Destination	Protocol	Length	Info
2	0.292232	192.168.43.114	110.40.71.61	TCP	66	58594 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
4	0.559207	110.40.71.61	192.168.43.114	TCP	66	80 → 58594 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1360 WS=256 SACK_PERM
5	0.559568	192.168.43.114	110.40.71.61	TCP	54	58594 → 80 [ACK] Seq=1 Ack=1 Win=131840 Len=0
6	0.827563	192.168.43.114	110.40.71.61	TCP	54	58594 → 80 [FIN, ACK] Seq=1 Ack=1 Win=131840 Len=0
8	1.071121	110.40.71.61	192.168.43.114	TCP	54	80 → 58594 [RST, ACK] Seq=1 Ack=2 Win=0 Len=0
26	3.036581	192.168.43.114	110.40.71.61	TCP	66	58619 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
27	3.037362	192.168.43.114	110.40.71.61	TCP	66	58620 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
28	3.289108	192.168.43.114	110.40.71.61	TCP	66	58621 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
29	3.304572	110.40.71.61	192.168.43.114	TCP	66	80 → 58620 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1360 WS=256 SACK_PERM
30	3.304953	192.168.43.114	110.40.71.61	TCP	54	58620 → 80 [ACK] Seq=1 Ack=1 Win=131840 Len=0
31	3.305589	192.168.43.114	110.40.71.61	HTTP	66	GET / HTTP/1.1
34	3.423057	110.40.71.61	192.168.43.114	TCP	66	80 → 58621 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1360 WS=256 SACK_PERM
35	3.423057	110.40.71.61	192.168.43.114	TCP	1414	80 → 58620 [ACK] Seq=1 Ack=615 Win=131840 Len=1360 [TCP segment of a reassembled PDU]
36	3.423057	110.40.71.61	192.168.43.114	HTTP	780	HTTP/1.1 200 OK (text/html)
37	3.423613	192.168.43.114	110.40.71.61	TCP	54	58621 → 80 [ACK] Seq=1 Ack=1 Win=131840 Len=0
38	3.423775	192.168.43.114	110.40.71.61	TCP	54	58620 → 80 [ACK] Seq=615 Ack=2087 Win=131840 Len=0
43	4.048021	192.168.43.114	110.40.71.61	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 58619 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
67	6.044236	192.168.43.114	110.40.71.61	TCP	66	[TCP Retransmission] [TCP Port numbers reused] 58619 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
70	6.110204	192.168.43.114	110.40.71.61	HTTP	66	GET / HTTP/1.1
71	6.403678	110.40.71.61	192.168.43.114	TCP	1414	80 → 58620 [ACK] Seq=2087 Ack=1229 Win=131072 Len=1360 [TCP segment of a reassembled PDU]

图 2 wireshark 捕获

点击某一条的 http 或者 tcp 追踪流，我们解析其中的部分报文：

(1) HTTP

分析 HTTP 的报文结构：

【请求报文】

GET / HTTP/1.1

Host: 2837428472.ysepan.com

Connection: keep-alive

Cache-Control: max-age=0

Upgrade-Insecure-Requests: 1

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36 Edg/133.0.0.0

Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

Accept-Encoding: gzip, deflate

Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

Cookie: ASP.NET_SessionId=fgpllhac45uxaylypcfkgps; ysl68-token=17446974692341902; jldl.2837428472=4.15

图 3 http 请求报文

第一行：请求行（请求方法和路径）

GET / HTTP/1.1

表示发送了一个 GET 请求，并表明使用 HTTP/1.1 版本。

第二行：请求对象主机

Host: 2837428472.ysepan.com

表示网页来源于域名 2837428472.ysepan.com，端口没有指定，默认 80 端口。

第三行：连接模式

Connection: keep-alive

Connection: keep-alive 表示客户端希望保持连接，以便可以发送多个请求。

这可以减少建立和关闭连接的开销，提高性能。

第四行：缓存控制

Cache-Control: max-age=0

Cache-Control 头字段用于指定缓存指令。max-age=0 指示浏览器不要使用缓存的响应，每次都请求最新的资源。这确保了用户总是获取最新的内容。

第五行：升级不安全请求

Upgrade-Insecure-Requests: 1

Upgrade-Insecure-Requests 是一个 HTTP 扩展头部，用于指示浏览器应该自动将不安全的 HTTP 请求升级到 HTTPS。这有助于提高安全性。

第六行：用户代理信息

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/133.0.0.0 Safari/537.36 Edg/133.0.0.0

User-Agent 头字段提供了客户端浏览器的信息，包括操作系统、浏览器类型和版本，表明客户端是一个运行在 Windows 10 上的 64 位系统上的 Chrome 浏览器。

第七行：可接受的内容的类型

Accept:

text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7

Accept 头字段指定客户端可以处理的内容类型，在这里，客户端优先接受 HTML、XHTML 和 XML 文档，但也可以接受其他类型的媒体，如图像和通用媒体类型。

第八行：可接受的内容编码

Accept-Encoding: gzip, deflate

客户端支持 **gzip** 和 **deflate** 压缩算法。

第九行：可接受的语言

Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6

客户端首选简体中文（zh-CN），其次是其他中文变体和英语。

第十行：客户端 Cookie 信息

Cookie:ASP.NET SessionId=fgpllhacb45uxaylypcfkgps;

ys168-token=17446974692341902;jldl.2837428472=4.15

头字段发送客户端存储的 Cookie。这些 Cookie 通常用于会话管理和身份验证。此示例中，发送了三个 Cookie，包括一个会话 ID 和一个身份验证令牌。

【响应报文】

HTTP/1.1 200 OK

Cache-Control: private

Content-Type: text/html; charset=utf-8

Server: Microsoft-IIS/7.5

X-AspNet-Version: 4.0.30319

X-Powered-By: ASP.NET

Access-Control-Allow-Origin: y.ys168.com:1001

```
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS
Date: Tue, 15 Apr 2025 07:17:48 GMT
Content-Length: 1697
```

图 4 http 响应报文

第一行：响应状态

HTTP/1.1 200 OK

HTTP/1.1 指定了使用的 HTTP 协议版本。200 OK 表示请求成功。状态码 200 是最常见的成功响应，表示服务器成功处理了请求并返回了预期的响应。

第二行：响应缓存控制

Cache-Control: private

说明指示响应只能被单个用户缓存（**private**）。这通常用于包含敏感用户数据的响应，以防止在共享缓存中存储。

第三行：内容类型

Content-Type: text/html; charset=utf-8

Content-Type 头字段指定响应内容的媒体类型。此示例中，响应内容是 HTML 文档。**charset=utf-8** 指定了字符编码，UTF-8 是一种广泛使用的字符编码，可以表示几乎所有字符。

第四行：服务器信息

Server: Microsoft-IIS/7.5

Server 头字段提供了服务器软件的信息，这里表示服务器使用的是 Microsoft Internet Information Services (IIS) 7.5。

第五行：ASP.NET 版本

X-AspNet-Version: 4.0.30319

`X-AspNet-Version` 是一个自定义头字段，提供了服务器端使用的 ASP.NET 框架版本。此示例中，使用的是 ASP.NET 4.0.30319。

第六行：服务器技术

`X-Powered-By: ASP.NET`

`X-Powered-By` 是一个自定义头字段，提供了服务器端使用的技术和框架信息，这一行说明了服务器端使用的是 ASP.NET。

第七行：允许跨域请求的源

`Access-Control-Allow-Origin: y.ys168.com:1001`

这个响应头是跨域资源共享（CORS）策略的一部分，它告诉浏览器哪些外部域可以访问资源。`y.ys168.com:1001` 指定了允许访问该资源的外部域和端口号。这意味着只有来自这个域和端口的 Web 应用才能通过浏览器请求这个资源。

第八行：允许跨域请求的头字段

`Access-Control-Allow-Headers: Content-Type, Authorization`

这个响应头指定了在跨域请求中，哪些自定义请求头可以被接受。

`Content-Type` 和 `Authorization` 是两个常见的 HTTP 头字段，分别用于指定请求体的媒体类型和提供认证信息。服务器通过这个响应头告诉客户端，这两个头字段在跨域请求中是被允许的。

第九行：允许跨域请求的方法

`Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS`

这个响应头指定了在跨域请求中，哪些 HTTP 方法可以被接受。GET、POST、PUT、DELETE 和 OPTIONS 是 HTTP 协议定义的一些标准方法，分别用于获

取资源、提交数据、更新资源、删除资源和描述通信选项。服务器通过这个响应头告诉客户端，这些方法在跨域请求中是被允许的。

第十行：响应日期和时间

Date: Tue, 15 Apr 2025 07:17:48 GMT

Date 响应头提供了响应生成的日期和时间。Tue, 15 Apr 2025 07:17:48 GMT 表示响应是在 2025 年 4 月 15 日星期二的协调世界时（GMT）早上 7 点 17 分 48 秒生成的。

第十一行：内容的长度

Content-Length: 1697

Content-Length 响应头指定了响应体的长度，单位是字节。1697 表示响应体包含 1697 个字节的数据。

十一行之后：响应内容体（如图 5 所示）

抓包、Web 服务

```

Content-Length: 1697

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      name="description"
      content=".....E....."
    />
    <link rel="icon" type="image/svg+xml" href="/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>.....E...</title>
    <script>
      window.htxx = {"qqdz":"http://c1.ysepan.com/nkj/", "dlmc":"2837428472", "ym":"ysepan.com"};
    </script>
    <script type="module" crossorigin src="/assets/index-BQyZTLr9.js"></script>
    <link rel="stylesheet" crossorigin href="/assets/index-daDQxGTf.css">
  </head>
  <body>
    <div id="app"></div>
    <div
      id="appjdt"
      style="
        position: fixed;
        width: 100%;
        text-align: center;
        top: 100px;
        color: #69e659;
        opacity: 0.6;
        transition: opacity 0.7s ease;
      "
    >
    <svg
      xmlns="http://www.w3.org/2000/svg"

```

图 5 响应内容体

(2) TCP

分析 TCP 的报文结构：

(以第一次握手为例，如图 6)：

```

> Frame 394: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface \Device\NPF_{F3E78951-A31A-4322-99DE-9326CBF21FB5}, id 0
> Ethernet II, Src: LiteonTe_9f:4a:d6 (e4:aa:ea:9f:4a:d6), Dst: f2:c3:3c:bb:0a:e6 (f2:c3:3c:bb:0a:e6)
> Internet Protocol Version 4, Src: 192.168.43.114, Dst: 110.40.71.61
v Transmission Control Protocol, Src Port: 64440, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 64440
  Destination Port: 80
  [Stream index: 3]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 1027990141
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 .... = Header Length: 32 bytes (8)
> Flags: 0x002 (SYN)
  Window: 64240
  [Calculated window size: 64240]
  Checksum: 0xb6d3 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
> Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted
> [Timestamps]

```

图 6 TCP 报文结构（以第一次握手为例）

如图所示，在捕获的帧中，显示了多个层次的信息：链路层（Ethernet II）、网络层（IPv4）、以及传输层（TCP）。对捕获的帧的解释如下：

Frame 394: 这是捕获的数据帧的基本信息，包括帧的大小（66 字节）、捕获的字节数（528 位），以及捕获接口的信息。

Ethernet II: （链路层信息）这一层提供了数据帧的源和目的 MAC 地址。

Internet Protocol Version 4 (IPv4): （网络层信息）这一层提供了数据包的源和目的 IP 地址。

Transmission Control Protocol (TCP): （传输层信息）这一层提供了 TCP 连接的源端口、目的端口、序列号等信息。

其中 Transmission Control Protocol (TCP)部分描述的是 TCP 报文段的具体内容，如图 7 所示：

```
Transmission Control Protocol, Src Port: 64440, Dst Port: 80, Seq: 0, Len: 0
  Source Port: 64440
  Destination Port: 80
  [Stream index: 3]
  [Conversation completeness: Incomplete, DATA (15)]
  [TCP Segment Len: 0]
  Sequence Number: 0 (relative sequence number)
  Sequence Number (raw): 1027990141
  [Next Sequence Number: 1 (relative sequence number)]
  Acknowledgment Number: 0
  Acknowledgment number (raw): 0
  1000 .... = Header Length: 32 bytes (8)
  > Flags: 0x002 (SYN)
  Window: 64240
  [Calculated window size: 64240]
  Checksum: 0xb6d3 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  > Options: (12 bytes), Maximum segment size, No-Operation (NOP), Window scale, No-Operation (NOP), No-Operation (NOP), SACK permitted
  > [Timestamps]
```



图 7 TCP 报文段的具体内容

一个 TCP 报文段的结构包含：源端口、目的端口、序列号、确认号、数据偏移、保留、标志位、窗口大小、校验和、紧急指针、选项、数据。

针对我们捕获到的报文段，我们进行**具体的分析**：

源端口：64440，即发送方（客户端）的端口号。

目的端口：80 这是接收方（服务器）的端口号，80 端口通常用于 HTTP 服务。

序列号：0，这是 TCP 连接的第一个数据包，序列号从 0 开始。

确认号 Ack：0，由于这是一个建立连接的 SYN 包，还没有收到对方的数据，所以确认号为 0。

数据长度 Len：0，表示这个 TCP 段没有携带数据，仅用于控制信息（如建立连接）。

标志位 Flags：根据图中信息，我们可以推断这是一个 SYN 包（建立连接的初始包），但图中没有直接显示标志位。通常，SYN 包的标志位会设置 SYN 标志。

窗口大小 Win：64240，表示接收方的缓冲区大小，用于流量控制。

校验和（Checksum）：0xb6d3 [unverified]，用于错误检测，确保数据的完整性。图中显示的状态是未验证（unverified）。

紧急指针（Urgent Pointer）：0，仅在 URG 标志位被设置时有效，此报文中没有设置 URG 标志。

仅在 URG 标志位被设置时有效。

选项（Options）：(12 bytes)，包含最大段大小（MSS）、窗口缩放（Window scale）、SACK 允许（SACK permitted）等选项。

➤ 数据（Data）：0 字节，此 TCP 段没有携带数据，仅用于控制信息（建立连接）。

分析 TCP 的工作方式：

394	3.785235	192.168.43.114	110.40.71.61	TCP	66 64440 → 80 [SYN] Seq=0 win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
406	3.893507	110.40.71.61	192.168.43.114	TCP	66 80 → 64440 [SYN, ACK] Seq=0 Ack=1 Win=8192 Len=0 MSS=1360 WS=256 SACK_PERM
412	3.894562	192.168.43.114	110.40.71.61	TCP	54 64440 → 80 [ACK] Seq=1 Ack=1 Win=131840 Len=0

图 8 tcp 工作方式（三次握手）

(a) 连接建立： TCP 使用三次握手过程来建立一个连接：

SYN: 客户端发送一个 SYN（同步序列编号）包到服务器以初始化一个连接。

SYN-ACK: 服务器收到 SYN 包后，回复一个 SYN-ACK 包，表示同意建立连接，并发送自己的初始序列号。

ACK: 客户端收到 SYN-ACK 包后，发送一个 ACK 包作为响应，完成三次握手，连接建立。

(b) 数据传输

数据分段： 发送方将数据分割成多个段，每个段都包含序列号，以确保数据的有序传输。

确认应答： 接收方收到数据段后，发送一个 ACK 包给发送方，确认数据已收到。如果发送方在一定时间内没有收到 ACK，它会重传数据。

校验和： 每个 TCP 段都包含一个校验和，用于检测数据在传输过程中是否发生了错误。如果校验和不匹配，数据将被丢弃。

(c) 连接终止： TCP 使用四次挥手过程来终止一个连接：

FIN: 一方发送 FIN（结束）包，表示它已经完成发送数据。

ACK: 另一方收到 FIN 包后，发送一个 ACK 包作为响应。

FIN: 然后另一方也发送一个 FIN 包，表示它也完成了发送数据。

ACK: 最初发送 FIN 的一方收到最后的 FIN 包后，发送一个 ACK 包，连接关闭。

(d) 在过程中，还涉及其他要素，如拥塞控制、重传机制等。

(五) 编程实现基于 TCP 协议的 Socket 通信。

```

1 package step1;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStreamReader;
6 import java.io.PrintWriter;
7 import java.net.ServerSocket;
8 import java.net.Socket;
9 import java.net.UnknownHostException;
10 import java.nio.charset.Charset;
11 import java.util.concurrent.CountDownLatch;
12 import java.util.concurrent.ExecutorService;
13 import java.util.concurrent.Executors;
14
15
16 public class SocketTest {
17     public static void main(String[] args) {
18         ExecutorService es = Executors.newFixedThreadPool(3);
19         CountDownLatch latch = new CountDownLatch(1); // 用于同步服务器启动 数字是锁的个数
20         // 提交服务端运行任务
21         es.submit(() -> {
22             try(ServerSocket server = new ServerSocket(4567);) { // 这个数字是端口，可以自己定
23                 latch.countDown(); // 通知客户端可以连接了
24                 Socket s = server.accept(); // 阻塞，监听到客户端请求后返回Socket对象
25                 //
26                 try(BufferedReader reader = new BufferedReader(new InputStreamReader(s.getInputStream(), Charset.forName("utf-8"))));{
27                     try(BufferedReader reader = new BufferedReader(new InputStreamReader(s.getInputStream(), Charset.forName("utf-8"))));{
28                         System.out.println("接收数据:" + reader.readLine());
29                     }
30                     catch (IOException e) {
31                         e.printStackTrace();
32                     }
33                 });
34                 try {
35                     latch.await(); // 阻塞直到服务器启动
36                 } catch (InterruptedException e) {
37                     e.printStackTrace();
38                 }
39                 // 提交客户端运行任务
40                 es.submit(() -> {
41                     try(Socket skt = new Socket("localhost", 4567); // 创建Socket对象，并尝试连接服务器端
42                         PrintWriter pw = new PrintWriter(skt.getOutputStream());){
43                         pw.println("你好"); // 发送文本数据
44                     } catch (UnknownHostException e) {
45                         e.printStackTrace();
46                     } catch (IOException e) {
47                         e.printStackTrace();
48                     }
49                 });
50                 es.shutdown();
51             }
52         });
53     }
54 }

```

图 9 基于 TCP 的 socket 通信

我们主要分析关键的部分（服务器端和客户端）：

1. 服务器端：

```

es.submit(() -> {
    try(ServerSocket server = new ServerSocket(4567)); {
        latch.countDown(); // 通知客户端可以连接了
        Socket s = server.accept(); // 阻塞, 监听到客户端请求后返回Socket对象
        try(BufferedReader reader = new BufferedReader(new
InputStreamReader(s.getInputStream(), Charset.forName("utf-8"))));{
            System.out.println("接收数据:" + reader.readLine());
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
});

```

图 10 服务器端相关代码

首先, 以类型 `ServerSocket` 创建了一个服务器端 `Socket`, 监听端口 4567。当服务器启动后, 调用 `countDown()` 方法, 通知客户端可以开始连接。然后, 在 `server.accept()` 处阻塞, 等待客户端连接。一旦有客户端连接, 返回一个 `Socket` 对象。接着通过 `InputStreamReader` 和 `BufferedReader` 读取客户端发送的数据。最后使用 `reader.readLine()` 读取客户端发送的一行数据并打印。

2. 客户端:

```

es.submit(() -> {
    try(Socket skt = new Socket("localhost", 4567)); // 创建Socket对象, 并尝试连接服务器端
    {
        PrintWriter pw = new PrintWriter(skt.getOutputStream());{
            pw.println("你好"); // 发送文本数据
        } catch (UnknownHostException e) {
            e.printStackTrace();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
});

```

图 11 客户端相关代码

首先, 使用 `Socket`, 让客户端尝试连接到服务器 (`localhost`, 端口 4567)。接着, `PrintWriter`: 通过 `PrintWriter` 向服务器发送文本数据。最后, `pw.println("你好")`: 发送字符串“你好”到服务器。

四. 实验结果分析

1.web 配置成功，能够打开编写的 html；

2. DNS 服务器的安装、配置、使用成功；

3. FTP 服务器的安装、配置与使用成功，能够打开相应位置的文件夹；

4.报文捕获、分析成功：用 wireshark 捕获到了相应的报文，并针对 http、tcp 报文结构、工作方式进行了分析。

5. 编程实现基于 TCP 协议的 Socket 通信：服务器能够收到来自客户端的“你好”，说明连接成功。

五. 实验相关问题回答

在上述实验的基础上，请回答以下问题：

1. 通过 ping 命令测试网络连通性的原理是什么？请举例给出 ping 命令的测试结果（允许附截图）。

Ping 命令原理：向指定网络发送一定长度的数据包，若指定网络地址存在，则返回同样大小的数据包，若在特定时间内没有返回，则是“超时”，认定为指定的网络地址不存在。

(1).发出 ICMP Echo 请求：运行命令之后，计算机会创建一个特殊的 ICMP Echo 请求包。这个数据包包含：特定的标识符和序列号，用于跟踪和识别请求、响应。

(2).发送请求数据包：发到目标主机的 ip 地址（计算机->路由器->交换机->...->目标主机）。

(3).目标主机进行响应：生成一个 ICMP Echo 响应数据包，返回给发送方。

(4).发送方接受响应数据包：ping 命令此时会记录响应包的延迟时间。
测试结果：如图 12 所示。

```
C:\Users\lenovot>Ping 2837428472.ysepan.com

正在 Ping 2837428472.ysepan.com [110.40.71.61] 具有 32 字节的数据:
来自 110.40.71.61 的回复: 字节=32 时间=232ms TTL=109
来自 110.40.71.61 的回复: 字节=32 时间=46ms TTL=109
来自 110.40.71.61 的回复: 字节=32 时间=50ms TTL=109
来自 110.40.71.61 的回复: 字节=32 时间=48ms TTL=109

110.40.71.61 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 4, 丢失 = 0 (0% 丢失),
往返行程的估计时间(以毫秒为单位):
    最短 = 46ms, 最长 = 232ms, 平均 = 94ms
```

图 12 ping 了一个网站（前面分析报文使用的网站）

2. 在交换机连接的计算机网络中，以太网地址（即 MAC 地址）的作用是什么？请写出你主机的 48 位 MAC 地址（允许附截图）。

作用：是网络设备在数据链路层的唯一标识符。在设备识别、数据帧转发、网络管理、安全控制等方面有重要作用。

48 位 MAC 地址：

```
以太网适配器 以太网:

媒体状态 . . . . . : 媒体已断开连接
连接特定的 DNS 后缀 . . . . . :
描述. . . . . : Realtek PCIe GbE Family Controller
物理地址. . . . . : F8-75-A4-55-CE-82
DHCP 已启用 . . . . . : 是
自动配置已启用. . . . . : 是
```

图 13 本机的 mac 地址

六. 实验报告评价

成绩:

指导教师:

日期: